

Clase 17

Tecnologías adicionales

IIC2513 - Tecnologías y Aplicaciones Web

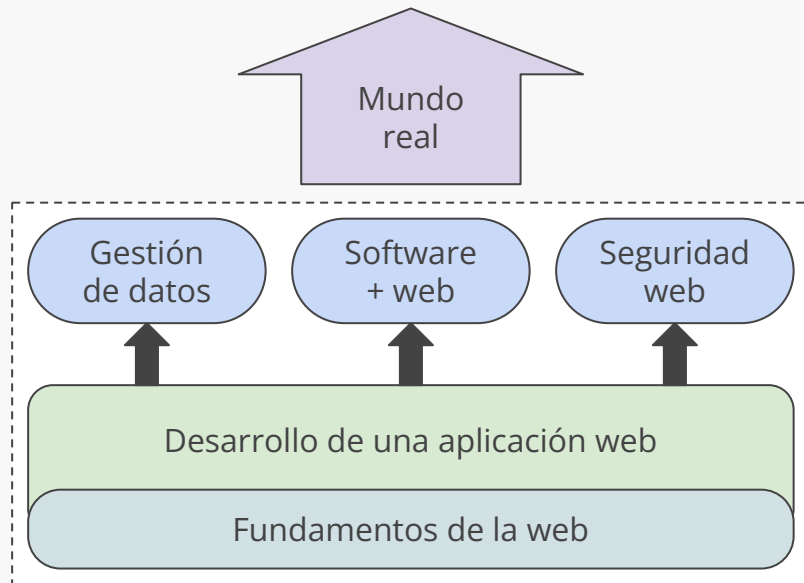
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. El viernes hay sala de ayuda (E1)

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

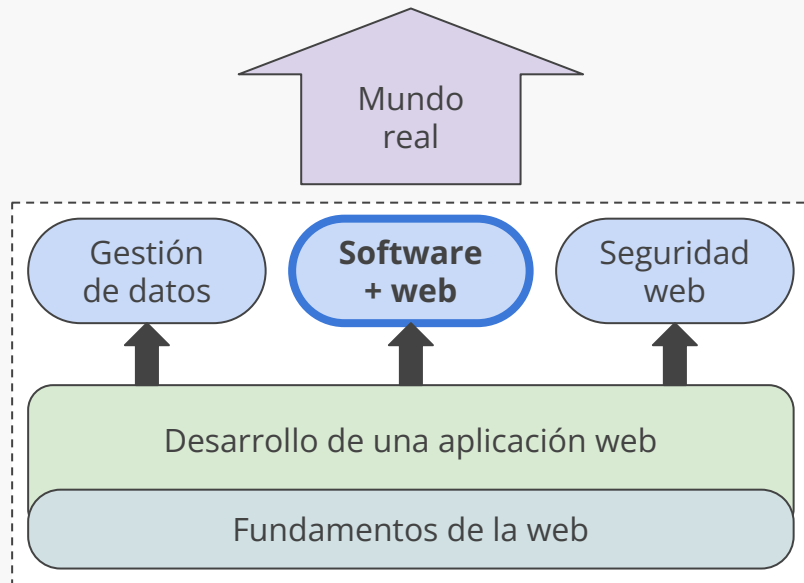


Contenidos del curso

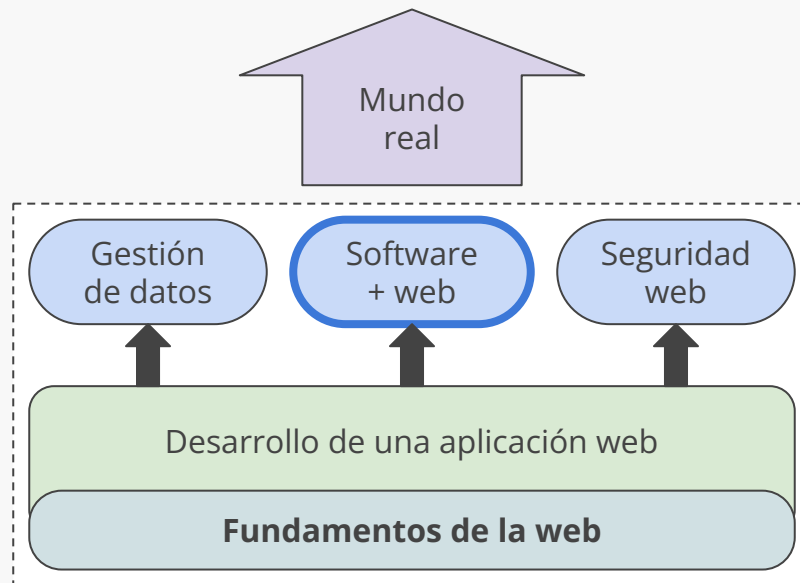
Software + web

Hablaremos sobre elementos que apuntan a expandir y **mejorar la experiencia de desarrollo web** tales como *testing* y *websockets*...

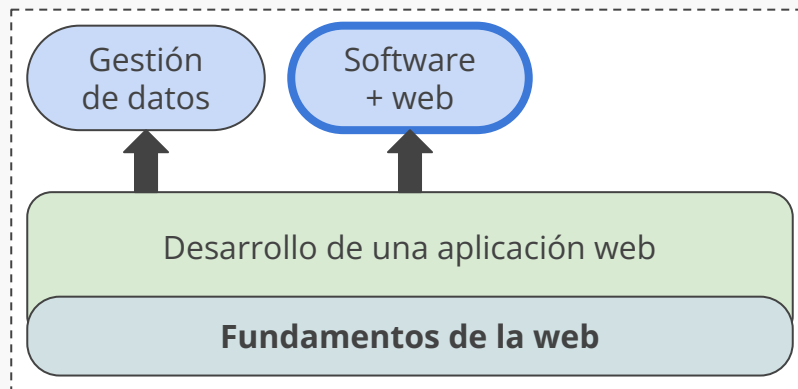
... para reconocer e identificar tecnologías de vanguardia y que permiten **buenas prácticas en desarrollo web**



Contenidos del curso



Contenidos del curso



Objetivo de la clase de hoy

- ❑ **Identificar** nuevas tecnologías para desarrollo web
- ❑ **Comparar** las funcionalidades nuevas posibles con una aplicación base

Menti#*Una pequeña encuesta*

¿Cómo van con la Entrega 1?



Sobre la clase anterior

Clase 16 - Almacenes de información:
Almacenamiento client-side y server-side

2



2



1



Menciona algo que hayas aprendido de esta clase

nada

no vine xd

Caché

INDEXDB

Nosql > sql

nueva planta

indexeddb es mejor que
sesskonstorage y
localStorage para algunas
cosas

Cache de cache

Menciona algo que hayas aprendido de esta clase

No tenía idea de todo lo que pasaron, así que aprendí mucho

esta o la anterior

no vine

ldb

indexdb

¿Cómo te fue con la Actividad 4? 🙄



¿Te quedó alguna duda o pregunta de esta clase?

depende quien
pregunta

tengo q estudiarlo por
mi cuenta

odio javascript

debería tener dudas o
preguntas acaso?



Sobre la clase de hoy

Clase 17 - Tecnologías adicionales

1



¿Hay alguna tecnología/librería/API web que quieras conocer?

Vue

NestJS

Tecnologías de
encriptado

no cacho mucho la
verdad de otras
opciones

una explicación más a
fondo de dummyJSON, ya
que lo usamos tanto para
las actividades

1



5



¡Gracias!

Agenda

✨ **TypeScript**

WebAssembly

WebSockets

Progressive Web Apps (PWAs)

¿Cómo es JavaScript como lenguaje?

- Es un lenguaje de *scripting*: **interpretado** como Python, es decir, “llegar y usar”
- Como todo lenguaje, JavaScript tiene partes buenas y no tan buenas (**¿mañas?**)
- Está en **constante extensión** (versión cada año)
- En sintaxis, es más **similar a C** que a otra cosa
- Es de **tipado dinámico**: variables tienen el tipo del último valor dado

A yellow square with a black border containing the letters 'JS' in a bold, black, sans-serif font.

¿Cómo es JavaScript como lenguaje?

Notar que JavaScript, para evitar errores producidos por conversión de tipos, intenta realizar conversiones apropiadas para llegar a un resultado

**Aunque JS no muestre errores...
hay que cuidarse de mezclar tipos
de datos**

```
1 "a" + 10;           // "a10"
2 "1" + 10;           // "110"
3 [] + 10;            // "10"
4 [2] + 10;           // "210"
5 [2, 2, 7] + 10;     // "2,2,710"
6 +"2" + 10;          // 12
7 -"1" + 10;          // 9
8 -"a" + 10;          // NaN
```

TypeScript

TypeScript (TS) es superset de JavaScript, desarrollado y mantenido por Microsoft, que añade tipos estáticos y objetos basados en clases

Implementa una formalización de los tipos de JavaScript, permitiendo definir variables y funciones tipadas



```
const user = {  
  firstName: 'Antonio',  
  lastName: 'Ossa',  
  role: 'Teacher',  
};
```

```
console.log(user.name);
```

```
Property 'name' does not exist on type '{ firstName: string;  
lastName: string; role: string; }'
```

TypeScript

Clase 3



TypeScript (TS) es superset de JavaScript

Clase 16#Tecnologías adicionales

Implementa una formalización de los tipos de JavaScript, permitiendo definir variables y funciones tipadas

```
firstName: 'ANTONIO',  
lastName: 'Ossa',  
role: 'Teacher',  
};
```

```
console.log(user.name);
```

```
Property 'name' does not exist on type '{ firstName: string;  
lastName: string; role: string; }'
```

TypeScript

TypeScript (TS) es superset de JavaScript, desarrollado y mantenido por Microsoft, que añade tipos estáticos y objetos basados en clases

Implementa una formalización de los tipos de JavaScript, permitiendo definir variables y funciones tipadas



```
const user = {  
  firstName: "Antonio",  
  lastName: "Ossa",  
  role: "Teacher",  
}
```

```
console.log(user.name)  
Property 'name' does not exist  
on type '{ firstName: string;  
lastName: string; role:  
string; }'.
```

TypeScript

- Desarrollado por Microsoft en 2012, versión 1.0 es del 2014. Es open source
- Es un superconjunto de JavaScript
- Elementos adicionales son convertidos a JavaScript (vía *transpiler*)
- Comenzó a ganar tracción cuando fué incorporado en el framework Angular como el lenguaje principal
- Ha ido aumentando la popularidad en los últimos años
- Actualmente en su versión 5.8



```
const user = {  
  firstName: "Antonio",  
  lastName: "Ossa",  
  role: "Teacher",  
}
```

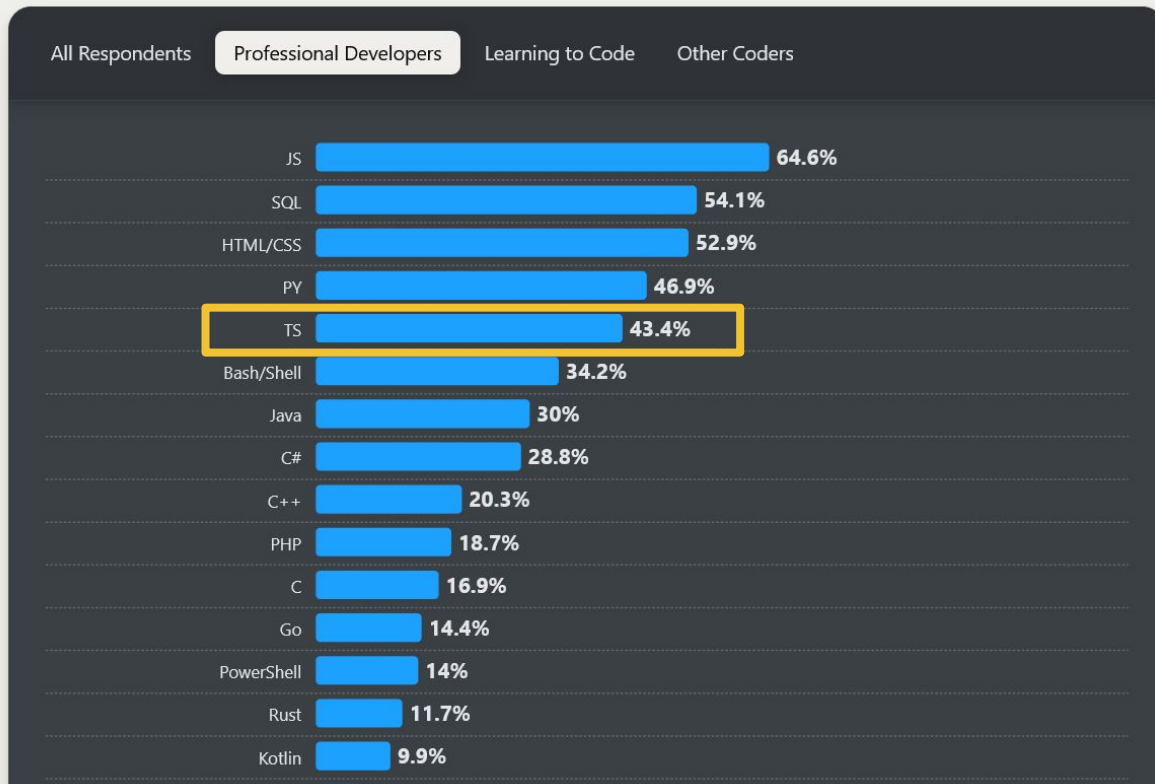
```
console.log(user.name)  
Property 'name' does not exist  
on type '{ firstName: string;  
lastName: string; role:  
string; }'.
```


Most popular technologies 2.1

Programming, scripting, and markup languages

JavaScript has been a mainstay in the developer survey and on Stack Overflow since our first survey. The most popular programming language has been JavaScript every year we have done the survey except for 2013 and 2014, when SQL was the most popular language.

? Which **programming, scripting, and markup languages** have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the language and want to continue to do so, please check both boxes in that row.)





Start a free trial

OCTOVERSE 2024

The state of open source software

In this year's Octoverse report, we study how public and open source activity on GitHub shows how AI is expanding as the global developer community surges in size.

Read now →



Start a free trial

OCTOVERSE 2024

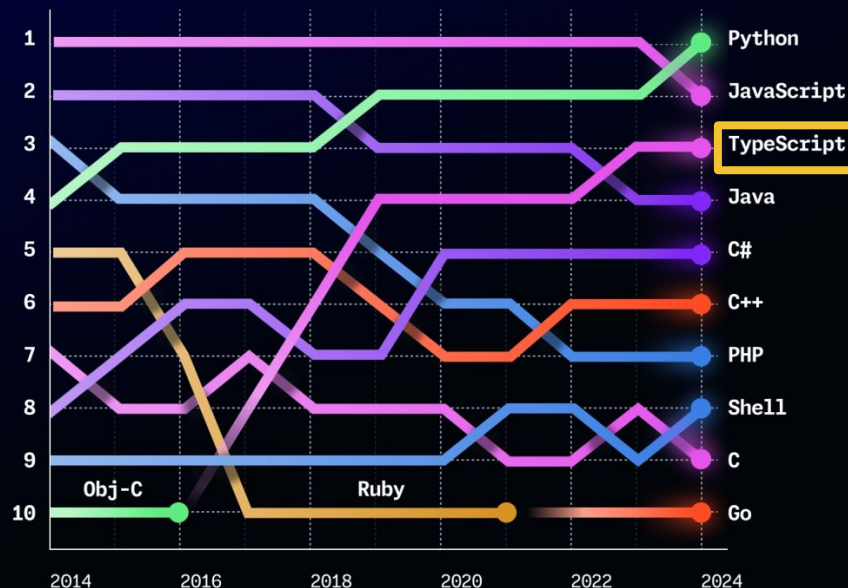
The state of open source

In this year's Octoverse on GitHub shows how AI surges in size.

Read now →

Top programming languages on GitHub

RANKED BY COUNT OF DISTINCT USERS CONTRIBUTING TO PROJECTS OF EACH LANGUAGE.



Principales beneficios

- *Type Safety*
 - *Static typing* permite detectar errores en tiempo de compilación*, lo que permite reducir errores y mejorar la calidad
- Escalabilidad
 - En proyectos grandes la mantención del código se facilita mucho
- *Tooling Support*
 - Autocompletación, *refactoring*, *code navigation*
 - Al pasar sobre una variable o función pueden saberse los detalles

TypeScript

Algunas de las features más destacadas de este lenguaje son:

- **Anotación de tipos (*type annotations*)**
- Inferencia de tipos (*type inference*)
- Declaración de interfaces (*interface*)
- Anotación de funciones (*function annotations*)

```
// Forma general
```

```
let variable: TypeName;
```

```
const CONSTANT: TypeName;
```

```
// Un ejemplo
```

```
let someValue: boolean = false;
```

```
let favNumber: number = 42;
```

```
let COURSES: string[] = ['IIC2513', 'IIC3585'];
```

```
favNumber = someValue;
```

```
// Type 'boolean' is not assignable to type 'number'.
```

```
COURSES = someValue;
```

```
// Type 'boolean' is not assignable to type 'string[]'.
```

TypeScript

Algunas de las features más destacadas de este lenguaje son:

- **Anotación de tipos (*type annotations*)**
- Inferencia de tipos (*type inference*)
- Declaración de interfaces (*interface*)
- Anotación de funciones (*function annotations*)

```
// Básicos: boolean, number, string
```

```
// any: usado para eliminar la restricción de tipo
var item1: any = { id: 1, name: 'item1' };
item1 = { id: 2 };
```

```
// unions : combinación de dos o mas tipos
function printObject(obj: string | number) {
    console.log(`obj = ${obj}`);
}
printObject(1);
printObject('string value');
```

```
// emums
enum DoorState {
    Open,
    Closed,
}
let doorStatus: DoorState = DoorState.Open;
```

TypeScript

Algunas de las features más destacadas de este lenguaje son:

- Anotación de tipos (*type annotations*)
- **Inferencia de tipos (*type inference*)**
- Declaración de interfaces (*interface*)
- Anotación de funciones (*function annotations*)

```
// Estas declaraciones son  
// EQUIVALENTES gracias a la  
// inferencia de tipos  
let nombre: string = "Antonio";  
let nombre = "Antonio";
```

```
// Estas declaraciones son  
// DISTINTAS gracias a la  
// inferencia de tipos  
let cantidad = 7;  
let cantidad: string = 7;  
  
// ¿Qué pasa en la segunda línea?
```

TypeScript

Algunas de las features más destacadas de este lenguaje son:

- Anotación de tipos (*type annotations*)
- Inferencia de tipos (*type inference*)
- **Declaración de interfaces (*interface*)**
- Anotación de funciones (*function annotations*)

```
// Forma general
interface InterfaceName {
  property1: Type1
  property2: Type2
}

// Un ejemplo
interface Point{
  x: number
  y: number
}

let start: Point = {
  x: 1,
  y: 2
}

let idk: Point = {
  x: 2
}

// Property 'y' is missing in type
// '{ x: number; }' but required in type 'Point'
```


TypeScript

Algunas de las features más destacadas de este lenguaje son:

- Anotación de tipos (*type annotations*)
- Inferencia de tipos (*type inference*)
- Declaración de interfaces (*interface*)
- **Anotación de funciones (*function annotations*)**

```
// Forma general
```

```
function fName(par1: Type1): ReturnType {  
    ...  
}
```

```
// Un ejemplo
```

```
function calculate(a: number, b: number): number {  
    return a + b + a * b + a ** b;  
}
```

```
console.log(calculate('3', '2'));
```

```
// error TS2345: Argument of type '"3"'
```

```
// is not assignable to parameter of type 'number'.
```

TypeScript:

Página oficial del lenguaje de programación TypeScript



Download Docs Handbook Community Playground Tools

Search Docs

TypeScript is JavaScript with syntax for types.

TypeScript is a strongly typed programming language that builds on JavaScript, giving you better tooling at any scale.

Try TypeScript Now
Online or via npm

Editor Checks Auto-complete Interfaces JSX

```
const user = {  
  firstName: "Angela",  
  lastName: "Davis",  
  role: "Professor",  
}
```

```
console.log(user.name)
```

Property 'name' does not exist on type '{ firstName: string; lastName: string; role: string; }'.

[TypeScript 5.8](#) is now available

What is TypeScript?

JavaScript and More

TypeScript adds additional syntax to JavaScript to support a **tighter integration with your editor**. Catch errors early in your editor.

A Result You Can Trust

TypeScript code converts to JavaScript, which **runs anywhere JavaScript runs**: In a browser, on Node.js, Deno, Bun and in your apps.

Safety at Scale

TypeScript understands JavaScript and uses **type inference to give you great tooling** without additional code.

TypeScript:

Página oficial del lenguaje de programación TypeScript



[Download](#) [Docs](#) [Handbook](#) [Community](#) [Playground](#) [Tools](#)

[Search Docs](#)

TypeScript is JavaScript with syntax for types.

TypeScript is a strongly typed programming language that builds on JavaScript, giving you better tooling at any

[Editor Checks](#) [Auto-complete](#) [Interfaces](#) [JSX](#)

```
const user = {  
  firstName: "Angela",  
  lastName: "Davis",  
  role: "Professor",  
}  
  
console.log(user.name)
```

¿Más features de TS?

[TypeScript 5.8](#) is now available

What is TypeScript?

JavaScript and More

TypeScript adds additional syntax to JavaScript to support a **tighter integration with your editor**. Catch errors early in your editor.

A Result You Can Trust

TypeScript code converts to JavaScript, which **runs anywhere JavaScript runs**: In a browser, on Node.js, Deno, Bun and in your apps.

Safety at Scale

TypeScript understands JavaScript and uses **type inference to give you great tooling** without additional code.

TypeScript - Duck Typing

```
var nameIdObject = { name: "myName", id: 1, print() { } };

nameIdObject = { id: 2, name: "anotherName", print() { } };
// El segundo objeto es del mismo tipo por sus propiedades

nameIdObject = { id: 3, name: "thirdName" };
// error TS2741: Property 'print' is missing in type '{ id: number; name: string; }'
// but required in type '{ name: string; id: number; print(): void; }'.

// En último objeto es de un tipo diferente
```

TypeScript - Casting explícito

```
// Puede usarse la palabra as  
let x: unknown = 'hello';  
console.log((x as string).length);
```

```
// Puede usarse < >  
let x = 'hello';  
console.log((<string>x).length);
```

TypeScript - Arrays

```
const names: string[] = [];  
names.push("Dylan"); // no error  
names.push(3);  
// Error: Argument of type 'number' is not assignable to parameter of type 'string'
```

```
const names: readonly string[] = ["Dylan"];  
names.push("Jack");  
// Error: Property 'push' does not exist on type 'readonly string[]'
```

```
const numbers = [1, 2, 3]; // inferred to type number[]  
numbers.push(4); // no error  
numbers.push("2");  
// Error: Argument of type 'string' is not assignable to parameter of type 'number'.  
let head: number = numbers[0]; // no error
```

TypeScript - Objetos

```
const car: { type: string; model: string; year: number } = {  
  type: 'Toyota',  
  model: 'Corolla',  
  year: 2009,  
};  
  
const car = {  
  type: 'Toyota',  
};  
car.type = 'Ford'; // no error  
car.type = 2;  
// Error: Type 'number' is not assignable to type 'string'.
```

TypeScript - Type aliases

```
type CarYear = number;
type CarType = string;
type CarModel = string;
type Car = {
  year: CarYear;
  type: CarType;
  model: CarModel;
};

const carYear: CarYear = 2001;
const carType: CarType = 'Toyota';
const carModel: CarModel = 'Corolla';
const car: Car = {
  year: carYear,
  type: carType,
  model: carModel,
};
```


TypeScript - Classes

```
class Person {  
  // Se puede usar private, public y protected  
  private name: string;  
  
  public constructor(name: string) {  
    this.name = name;  
  }  
  
  public getName(): string {  
    return this.name;  
  }  
}  
  
const person = new Person("Jane");  
console.log(person.getName());  
// person.name isn't accessible  
// from outside the class since it's private
```

TypeScript - Más sobre clases

```
// Constructor permite inicializar los objetos en forma segura
class ClassWithConstructor {
  id: number;
  constructor(_id: number) {
    this.id = _id;
  }
}

// Pueden asignarse modificadores public, private a las propiedades o métodos de una clase
class ClassWithPrivateProperty {
  private id: number;
  constructor(id: number) {
    this.id = id;
  }
}

let privateAccess = new ClassWithPrivateProperty(10);
privateAccess.id = 20;
// error TS2341: Property 'id' is private and only accessible within class 'ClassWithPrivateProperty'
```

TypeScript - Más sobre clases

```
class Person {  
  // name is a private member variable  
  public constructor(private name: string) {}  
  
  public getName(): string {  
    return this.name;  
  }  
}  
const person = new Person("Jane");  
console.log(person.getName());
```

```
class Person {  
  private readonly name: string;  
  
  public constructor(name: string) {  
    // name cannot be changed after this  
    // initial definition, which has to be  
    // either at it's declaration  
    // or in the constructor.  
    this.name = name;  
  }  
  public getName(): string {  
    return this.name;  
  }  
}  
  
const person = new Person("Jane");  
console.log(person.getName());
```

TypeScript - Classes implement interfaces

```
interface Shape {  
  getArea: () => number;  
}  
  
class Rectangle implements Shape {  
  public constructor(  
    protected readonly width: number,  
    protected readonly height: number  
  ) {}  
  
  public getArea(): number {  
    return this.width * this.height;  
  }  
}
```

TypeScript - Herencia

```
interface Shape {  
    getArea: () => number;  
}  
  
class Rectangle implements Shape {  
    public constructor(  
        protected readonly width: number,  
        protected readonly height: number  
    ) {}  
  
    public getArea(): number {  
        return this.width * this.height;  
    }  
}  
  
class Square extends Rectangle {  
    public constructor(width: number) {  
        super(width, width);  
    }  
    // getArea gets inherited from Rectangle  
}
```

TypeScript - Override explícito

```
interface Shape {
  getArea: () => number;
}

class Rectangle implements Shape {
  // using protected for these members allows access
  // from classes that extend from this class, such as Square
  public constructor(
    protected readonly width: number,
    protected readonly height: number
  ) {}

  public getArea(): number {
    return this.width * this.height;
  }

  public toString(): string {
    return `Rectangle[width=${this.width}, height=${this.height}]`;
  }
}
```

```
class Square extends Rectangle {
  public constructor(width: number) {
    super(width, width);
  }

  // this toString replaces
  // the toString from Rectangle
  public override toString(): string {
    return `Square[width=${this.width}]`;
  }
}
```

TypeScript - Herencia

- Una clase puede extender a otra e implementar a una tercera
- Solo se puede extender una clase (no hay herencia múltiple) pero se puede implementar más de una interfaz

```
class BaseClass implements IBase {  
    id: number = 0;  
}  
class DerivedFromBaseClass extends BaseClass  
implements IDerivedFromBase {  
    name: string = 'nameString';  
}
```

TypeScript - *Generics*

En los lenguajes con tipos estáticos (C++) es importante para poder implementar funciones más generales. Por ejemplo una función que entregue los elementos de una lista concatenados:

- `[1, 2, 3] => "1, 2, 3"`
- `["uno", "dos", "tres"] => "uno, dos, tres"`

```
function printGeneric<T>(value: T) {  
  console.log(`typeof T is : ${typeof value}`);  
  console.log(`value is : ${value}`);  
}  
/*  
printGeneric(1);  
  typeof T is : number  
  value is : 1  
  
printGeneric(true);  
  typeof T is : boolean  
  value is : true  
  
printGeneric(() => { });  
  typeof T is : function  
  value is : function () { }  
  
printGeneric({ id: 1 });  
  typeof T is : object  
  value is : [object Object]  
*/
```


TypeScript - *Generics*

```
class NamedValue<T> {  
  private _value: T | undefined;  
  
  constructor(private name: string) {}  
  
  public setValue(value: T) {  
    this._value = value;  
  }  
  
  public getValue(): T | undefined {  
    return this._value;  
  }  
  
  public toString(): string {  
    return `${this.name}: ${this._value}`;  
  }  
}  
  
let value = new NamedValue<number>('myNumber');  
value.setValue(10);  
console.log(value.toString()); // myNumber: 10
```

Agenda

TypeScript

✨ **WebAssembly**

WebSockets

Progressive Web Apps (PWAs)

WebAssembly

WebAssembly (o wasm) es un **nuevo formato de código binario** desarrollado para ser ejecutado por navegadores modernos

- Es un **nuevo lenguaje**
El “cuarto lenguaje de la web”
- Es **binario**
No es para escribirlo, es un *compilation target*
- Es de **bajo nivel**
Instrucciones en código de máquina
- Es **para la web**
Diseñado para complementar otras tecnologías web



WebAssembly

WebAssembly (Wasm)
desarrollado por el consorcio

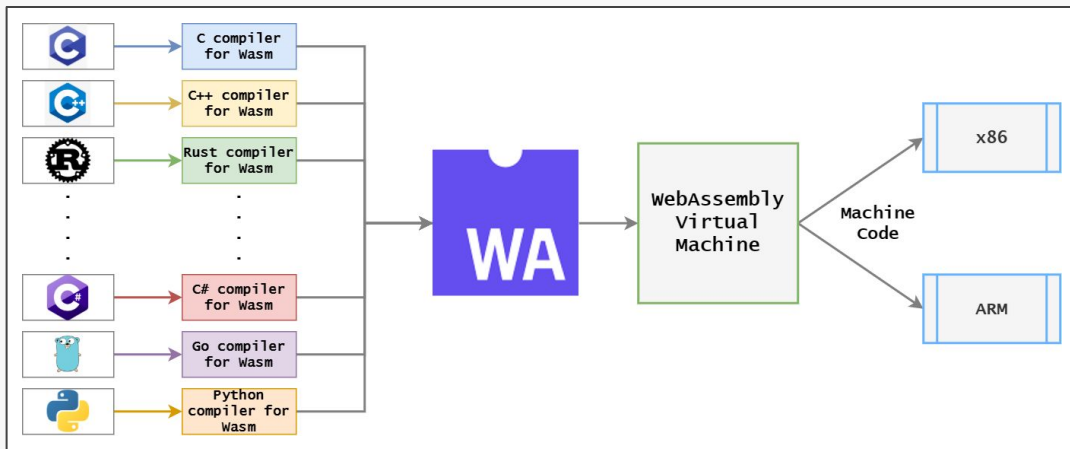
- Es un **nuevo** lenguaje de programación. El "cuarto lenguaje de programación" de la web.
- Es **binario**. No es para escribir código, sino para ejecutarlo.
- Es de **bajo nivel**. Instrucciones de bajo nivel.
- Es **para la web**. Diseñado para ser usado en navegadores.

0061	736d	0100	0000	0192	0116	6003	7f7f
7f01	7f60	067f	7c7f	7f7f	7f01	7f60	027f
7f00	6003	7f7e	7f01	7e60	017f	017f	6004
7f7f	7f7f	017f	6005	7f7f	7f7f	7f01	7f60
017f	0060	0000	6004	7f7e	7f7f	017f	6002
7f7f	017f	6000	017f	6002	7c7f	017c	6007
7f7f	7f7f	7f7f	7f01	7f60	037f	7f7f	0060
047f	7f7f	7f00	6003	7e7f	7f01	7f60	027e
7f01	7f60	057f	7f7f	7f7f	0060	017c	017e
6004	7f7e	7e7f	0060	027e	7e01	7c02	9702
0c03	656e	7608	5344	4c5f	496e	6974	0004
0365	6e76	1053	444c	5f53	6574	5669	6465



WebAssembly

WebAssembly (o wasm) es un **formato de código binario** que permite ejecutar código de alto rendimiento en navegadores web, por lo que lenguajes como **C, Rust o Python se pueden compilar a instrucciones para el navegador** de forma eficiente, simplemente traduciendo sus instrucciones regulares a este formato de instrucciones



¿Qué ofrece WebAssembly (Wasm)?

- **Eficiente y rápido**

Formato para máquina virtual (*stack based*)

Target portable para compiladores de alto nivel de C/C++/Rust

- **Cliente y servidor**

Corre código en el navegador a velocidad de código nativo

Puede usarse en el lado del cliente o del servidor (WASI)

- **Seguro**

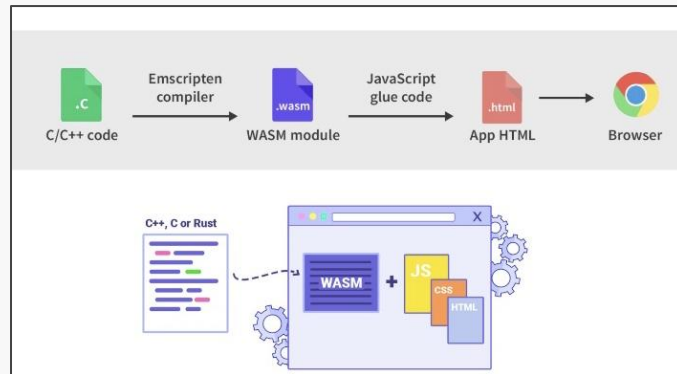
Módulos Wasm pueden ser incorporados en JS

Seguro (*sandboxed*)

- **Estándar**

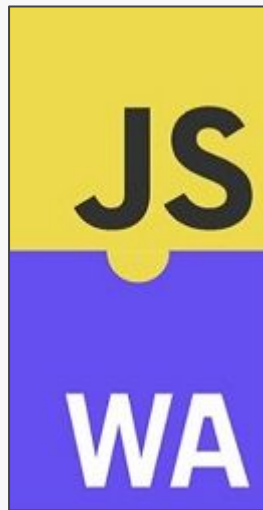
Estándar desarrollado por el W3C Web Assembly Working Group

Soportado en navegadores modernos



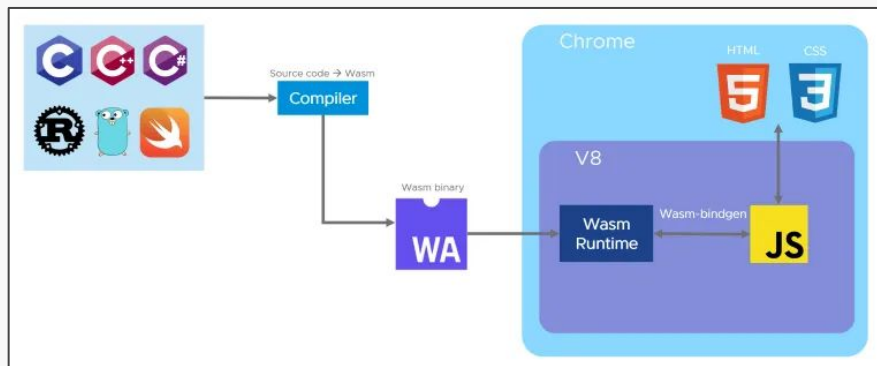
¿Por qué es tan importante?

- Puede alcanzar mejor *performance* que JavaScript
Paquetes más pequeños, código ejecutado más rápido
- Reuso de código
No solo librerías de JS sino que código en C, C++ y otros lenguajes conviven
- Versatilidad
Puede ser usado en una gran variedad de escenarios
- Interoperabilidad
Módulos en Wasm originados en diferentes lenguajes pueden interoperar fácilmente
- Estandarización
Formato binario estandarizado soportado por el W3C
- Independencia de la plataforma
No importa el sistema operativo ni el navegador, incluso no requiere navegador (WASI)



No es un reemplazo a JavaScript

- JavaScript es de alto nivel, muy flexible y con un gran ecosistema
- JS interactúa muy bien con el DOM, y lo usamos para cargar Wasm
- JS ya es muy rápido (no al nivel de C o C++ pero es suficiente)
- Para compilar JS a Wasm habría que limitar enormemente al primero
- Wasm es de bajo nivel con desempeño cercano a código nativo, permite programar en lenguajes como C++ o Rust



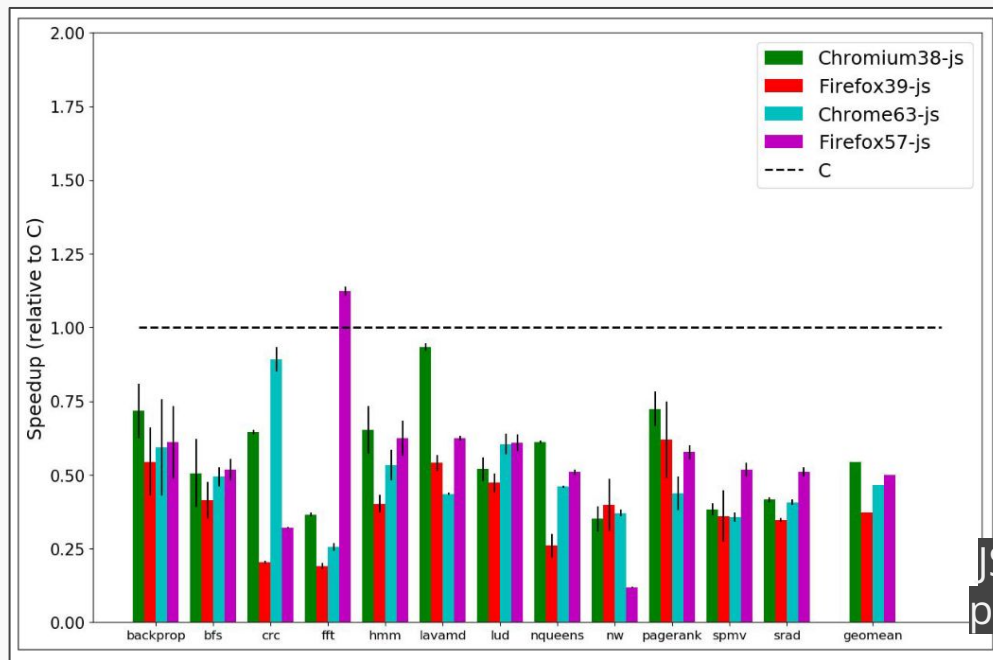
Streaming compilation

- No se requiere descargar todo el código para comenzar a compilar
- El navegador es capaz de compilar y generar los *bytecodes* de Wasm mientras es descargado
- Crucial para las aplicaciones en que el código corre en el lado del navegador



No es un reemplazo a JavaScript

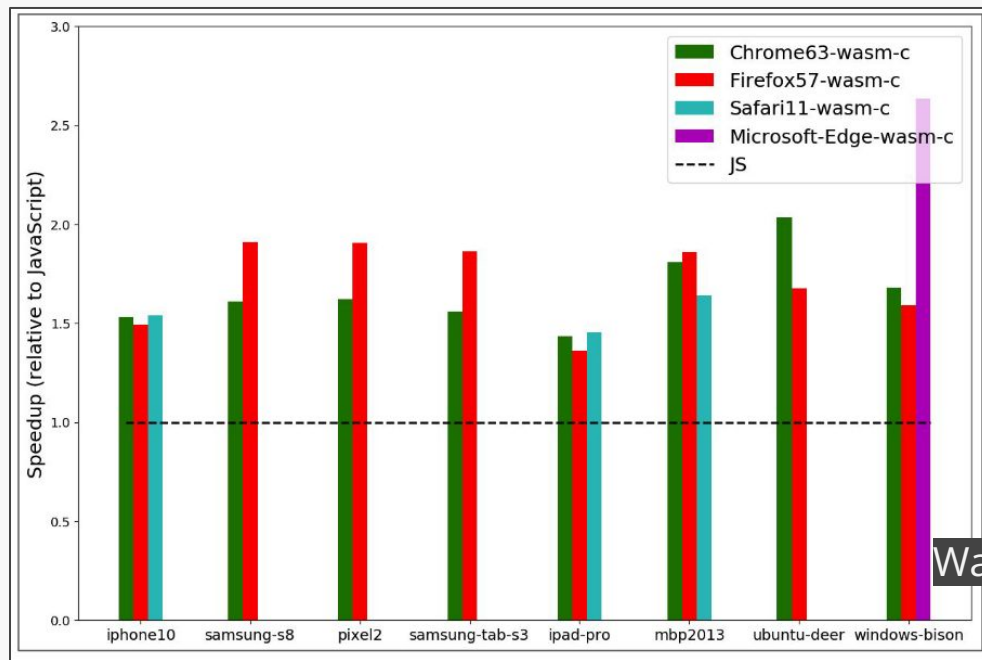
- JS ya es muy rápido (no al nivel de C o C++ pero es suficiente)



JS está lejos de la performance de C

No es un reemplazo a JavaScript

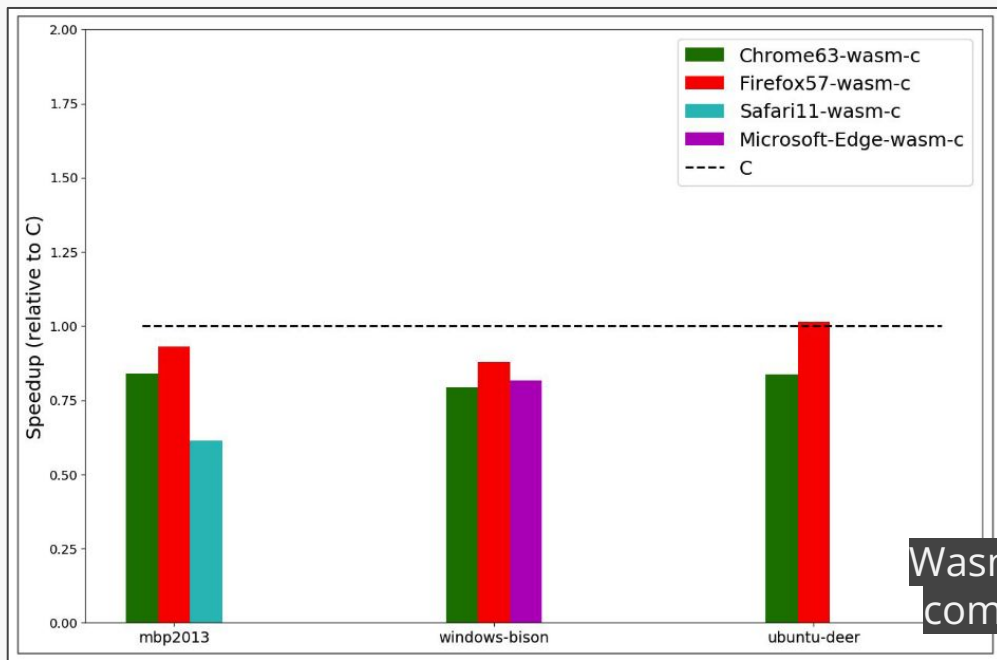
- Wasm es de bajo nivel con desempeño cercano a código nativo



Wasm es mucho más rápido que JS

No es un reemplazo a JavaScript

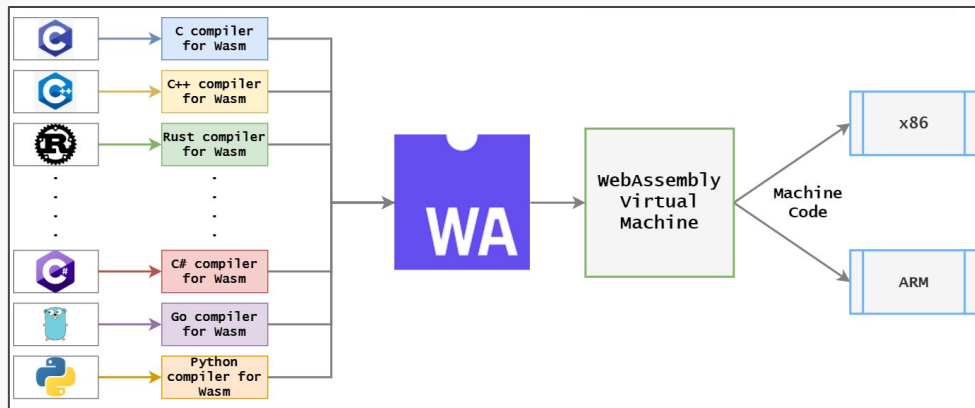
- Wasm es de bajo nivel con desempeño cercano a código nativo



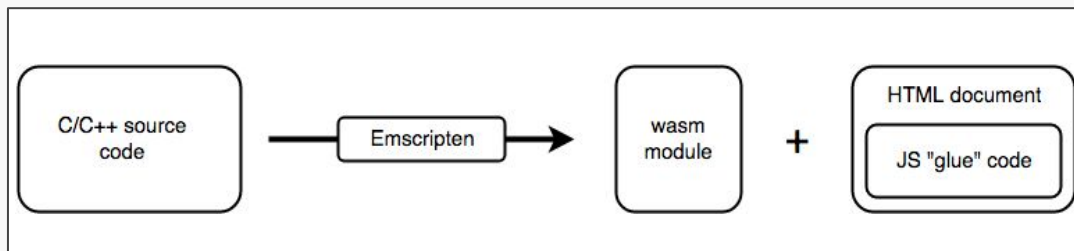
Wasm es casi tan rápido
como C (código nativo)

¿Cómo funciona?

1. Los desarrolladores escriben su código en el lenguaje de alto nivel de su elección
 2. Código es compilado a *wasm* *bytecodes* (*target*)
 3. Los *bytecodes* son ejecutados en el *browser* (*high speed*)
- + Más rápido en carga y ejecución
 - + Safe sandboxed execution (a-la-JS)



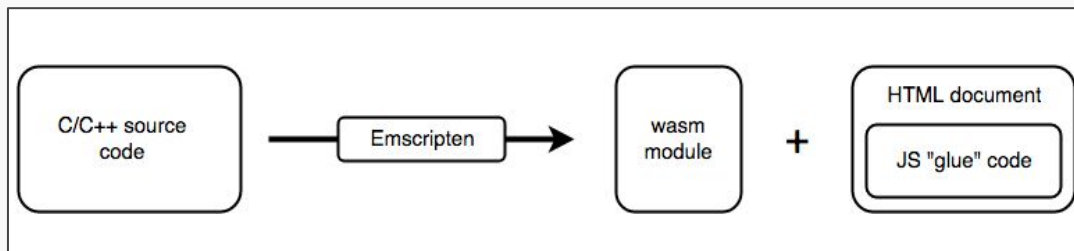
¿Cómo funciona?



```
1 #include <stdio.h>
2
3 int main() {
4     printf("hello, world!\n");
5     return 0;
6 }
7
```

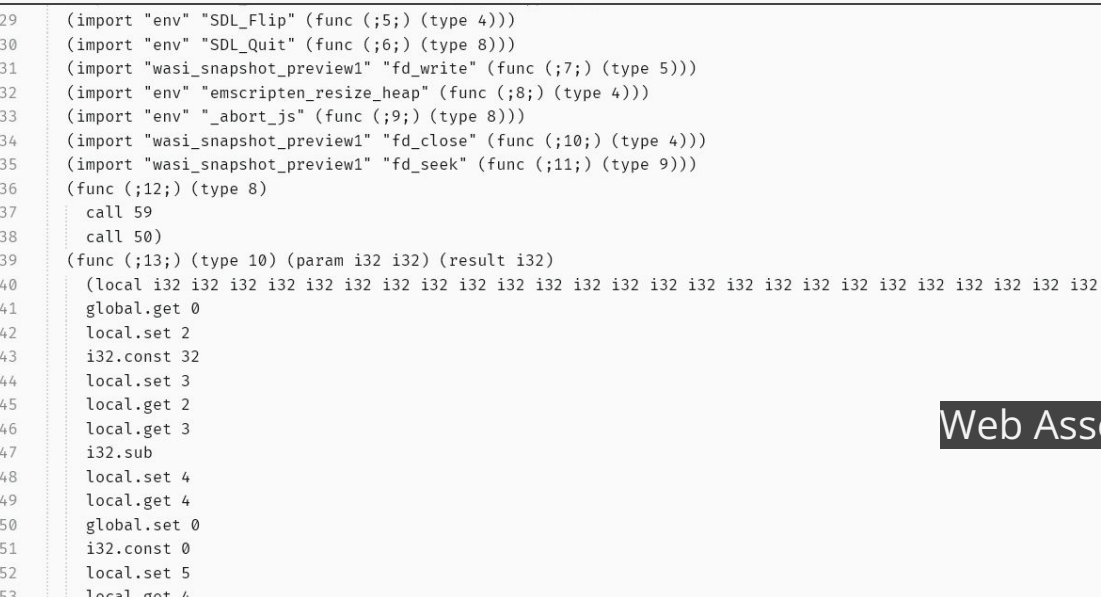
Código en C

¿Cómo funciona?



```
1  0061 736d 0100 0000 0192 0116 6003 7f7f
2  7f01 7f60 067f 7c7f 7f7f 7f01 7f60 027f
3  7f00 6003 7f7e 7f01 7e60 017f 017f 6004
4  7f7f 7f7f 017f 6005 7f7f 7f7f 7f01 7f60
5  017f 0060 0000 6004 7f7e 7f7f 017f 6002
6  7f7f 017f 6000 017f 6002 7c7f 017c 6007
7  7f7f 7f7f 7f7f 7f01 7f60 037f 7f7f 0060
8  047f 7f7f 7f00 6003 7e7f 7f01 7f60 027e
9  7f01 7f60 057f 7f7f 7f7f 0060 017c 017e
10 6004 7f7e 7e7f 0060 027e 7e01 7c02 9702
11 0c03 656e 7608 5344 4c5f 496e 6974 0004
```

Web Assembly (WASM)



Web Assembly Text (WAT)

Casos de uso

- Aplicaciones Web de alto desempeño (simulaciones, cálculos complejos, graphics, etc)
- Aplicaciones móviles híbridas y nativas (PWA con Wasm)
- Aplicaciones con código complejo en el lado del servidor
- Intérpretes de lenguajes y máquinas virtuales
- Procesamiento de multimedia
- Encriptación
- Simulaciones científicas y visualizaciones
- ... y muchos más por descubrir

Usos importantes hoy

- **Tensorflow.js**: 10x performance sobre plain JS ([docs](#))
- **Unity** (videogames): Hace posible exportar a la web juegos 2D y 3D más fácil ([docs](#))
- **AutoCAD**: Código fuente en C/C++ (>35 años) portado al navegador ([charla](#))
- **Google Earth**: Utilizando código en C/C++ para generar gráficos 3D ([artículo](#))

Pero... aún no ha tenido la masividad esperada

Made with WebAssembly:

Lista de proyectos que utilizan WebAssembly



Made with WebAssembly

[Home](#)

[New Projects](#)

[About](#)

Search for a use case, technology, or project:

E.g Games, Rust, AssemblyScript, Wasm By Example, ...

134 projects



Pion

Pure Go implementation of the WebRTC API



pprmt.cc

Secure MP3 Encoder



Prism

Build frontend web apps with Ruby



1Password

1Password used WebAssembly to speed up their browser plugin

BOBDEKit

Emscripten: Compilador de C y C++ a WebAssembly que entrega código base

**emscripten**[Introducing Emscripten](#)[Getting Started](#)[Compiling and Running Projects](#)[Porting](#)[API Reference](#)[Tools Reference](#)[Optimizing Code](#)[Optimizing WebGL](#)[Debugging with Sanitizers](#)[Building Emscripten from Source](#)[Contributing to Emscripten](#)[Profiling the Toolchain](#)[About this site](#)[Index](#)[Documentation](#)[Downloads](#)[Community](#)[Home » Main](#)[Fork me on GitHub](#)**emscripten**

Emscripten is a complete compiler toolchain to WebAssembly, using LLVM, with a special focus on speed, size, and the Web platform.

Porting

Compile your existing projects written in C or C++ — or any language that uses [LLVM](#) — to browsers, [Node.js](#), or [wasm runtimes](#).

APIs

Emscripten converts OpenGL into WebGL, and has support for familiar APIs like SDL, pthreads, and POSIX, as well as Web APIs and JavaScript.

Fast

Thanks to the combination of LLVM, Emscripten, [Binaryen](#), and [WebAssembly](#), the output is compact and runs at near-native speed.

Interested to learn more? Read our [About Page](#)!

[Instala](#)[Aprende](#)[Playground](#)[Herramientas](#)[Gobernanza](#)[Comunidad](#)[Blog](#)[Español \(es\)](#)

Rust

El lenguaje que empodera a todos
para construir software fiable y eficiente.

[COMIENZA](#)[Versión 1.86.0](#)

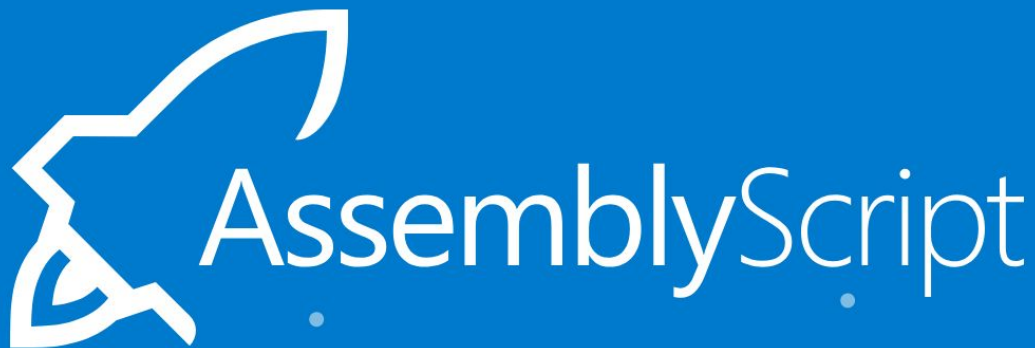
¿Por qué Rust?

Rendimiento

Fiabilidad

Productividad

AssemblyScript: Lenguaje tipo TypeScript para desarrollar en WebAssembly



A TypeScript-like language for WebAssembly.



Get Started



GitHub



npm

Designed for WebAssembly

Familiar TypeScript syntax

Right at your fingertips

Agenda

TypeScript

WebAssembly

✨ **WebSockets**

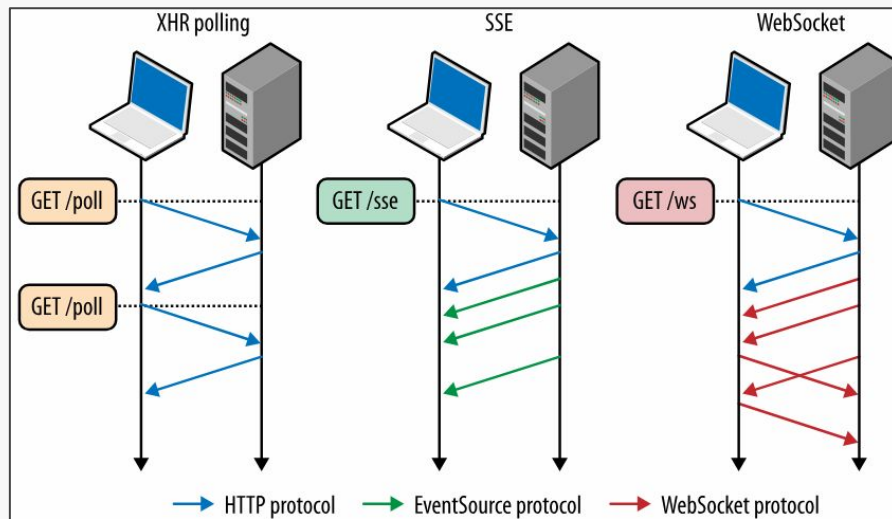
Progressive Web Apps (PWAs)

Comunicación web

APIs sobre comunicación y transferencia de datos. Enfocadas en permitir interacciones eficientes entre cliente y servidor o entre clientes, según distintos requerimientos

Elementos que deben conocer:

- Fetch API
- WebSocket API
- Server-Sent Events (SSE)
- WebRTC



Comunicación web

- La [Fetch API](#) es la forma moderna y flexible de realizar solicitudes HTTP desde el navegador y con soporte para Promesas
- La [WebSocket API](#) permite establecer una conexión persistente y bidireccional entre cliente y servidor, ideal para aplicaciones en tiempo real como chats o notificaciones
- Los [Server-Sent Events \(SSE\)](#) ofrecen un canal unidireccional desde el servidor al cliente para enviar actualizaciones constantes
- [WebRTC API](#) está diseñada para establecer comunicaciones en tiempo real (audio, video o datos) directamente entre navegadores u otros dispositivos, sin intermediarios

Comunicación web

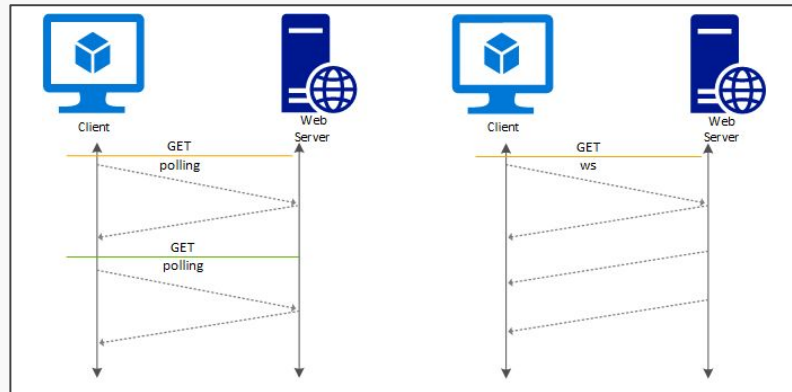
- La [Fetch API](#) es la forma moderna y flexible de realizar solicitudes HTTP desde el navegador y con soporte para Promesas
- La [WebSocket API](#) permite establecer una conexión persistente y bidireccional entre cliente y servidor, ideal para aplicaciones en tiempo real como chats o notificaciones
- Los [Server-Sent Events \(SSE\)](#) ofrecen un canal unidireccional desde el servidor al cliente para enviar actualizaciones constantes
- [WebRTC API](#) está diseñada para establecer comunicaciones en tiempo real (audio, video o datos) directamente entre navegadores u otros dispositivos, sin intermediarios

WebSockets: comunicación bidireccional

WebSockets es un **protocolo de comunicación** que permite la **interacción bidireccional en tiempo real entre un cliente y un servidor**. A diferencia de HTTP, WebSockets mantiene una conexión persistente (utilizando un *socket* TCP)

Algunas aplicaciones:

- Chats en vivo
- Realizar *broadcasting* de información (ej: actualizar marcadores deportivos en vivo)
- Sincronizar datos entre clientes (ej: Menti)
- Colaboración entre múltiples usuarios (ej: Figma)



The WebSocket API (WebSockets) - Web APIs | MDN:

Documentación de la API de WebSockets (interacción directa con API)

Filter

The WebSocket API (WebSockets)

Guides

Writing WebSocket client applications

Writing WebSocket servers

Writing a WebSocket server in C#

Writing a WebSocket server in Java

Writing a WebSocket server in JavaScript (Deno)

Using WebSocketStream to write a client

Interfaces

WebSocket

WebSocketStream

CloseEvent

MessageEvent

The WebSocket API (WebSockets)

Note: This feature is available in [Web Workers](#).

The **WebSocket API** makes it possible to open a two-way interactive communication session between the user's browser and a server. With this API, you can send messages to a server and receive responses without having to poll the server for a reply.

The WebSocket API provides two alternative mechanisms for creating and using web socket connections: the [WebSocket](#) interface and the [WebSocketStream](#) interface.

- The [WebSocket](#) interface is stable and has good browser and server support. However it doesn't support [backpressure](#). As a result, when messages arrive faster than the application can process them it will either fill up the device's memory by buffering those messages, become unresponsive due to 100% CPU usage, or both.
- The [WebSocketStream](#) interface is a [Promise](#)-based alternative to [WebSocket](#). It uses the [Streams API](#) to handle receiving and sending messages, meaning that socket connections can take advantage of stream backpressure automatically, regulating the speed of reading or writing to avoid bottlenecks in the application. However, [WebSocketStream](#) is non-standard and currently only supported in one rendering engine.

Additionally, the [WebTransport API](#) is expected to replace the WebSocket API for many applications. WebTransport is a versatile, low-level API that provides backpressure and many other features not supported by either [WebSocket](#) or [WebSocketStream](#), such as unidirectional streams, out-of-order delivery, and unreliable data transmission via datagrams. WebTransport is more complex to use than WebSockets and its cross-browser support is not as wide, but it enables the implementation of

In this article

Interfaces

Related HTTP headers

Guides

Tools

Specifications

Browser compatibility

See also

WebSockets: server

El server crea un *WebSocket server* usando la librería *ws* (Node.js) y escucha el puerto 8080

Cuando el cliente se conecta al websocket server el servidor, define *listeners* para recibir mensajes y detectar desconexión

Cuando el server recibe un mensaje del cliente, se lo envía de regreso

```
const WebSocket = require('ws');

// Create a WebSocket server
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', ws => {
  console.log('Client connected');

  // Event listener for receiving messages
  ws.on('message', message => {
    console.log(`Received: ${message}`);

    // Echo the message back to the client
    ws.send(`Echo: ${message}`);
  });

  // Event listener for disconnection
  ws.on('close', () => {
    console.log('Client disconnected');
  });
});
```

WebSockets: *client*

El cliente crea una conexión al WebSocket `ws://localhost:8080`, envía un mensaje al *server* cuando la conexión es abierta y loguea mensajes recibidos desde el server

Cuando el cliente es desconectado del WebSocket *server*, loguea un mensaje indicando desconexión

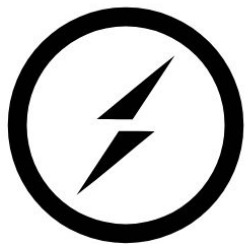
```
<!DOCTYPE html>
<html>
  <head>
    <title>WebSocket Example</title>
  </head>
  <body>
    <script>
      // Create a WebSocket connection
      const ws = new WebSocket('ws://localhost:8080');

      // Event listener for connection open
      ws.addEventListener('open', () => {
        console.log('Connected to WebSocket server');

        // Send a message to the server
        ws.send('Hello, WebSocket server!');
      });

      // Event listener for receiving messages
      ws.addEventListener('message', (event) => {
        console.log(`Received: ${event.data}`);
      });

      // Event listener for connection close
      ws.addEventListener('close', () => {
        console.log('Disconnected from WebSocket server');
      });
    </script>
  </body>
</html>
```



Socket.IO

Bidirectional and low-latency communication for every platform

[Get started](#)[Documentation](#)

Performant

In most cases, the connection will be established with WebSocket, providing a low-overhead communication channel between the server and the client.



Reliable

Rest assured! In case the WebSocket connection is not possible, it will fall back to HTTP long-polling. And if the connection is lost, the client will automatically try to reconnect.



Scalable

Scale to multiple servers and send events to all connected clients with ease.

Rooms | Socket.IO:

Documentación sobre Rooms (objeto que permite agrupar sockets de clientes)

Documentation

Server

Client

Events

Emitting events

Listening to events

Broadcasting events

Rooms

Adapters

Advanced

Migrations

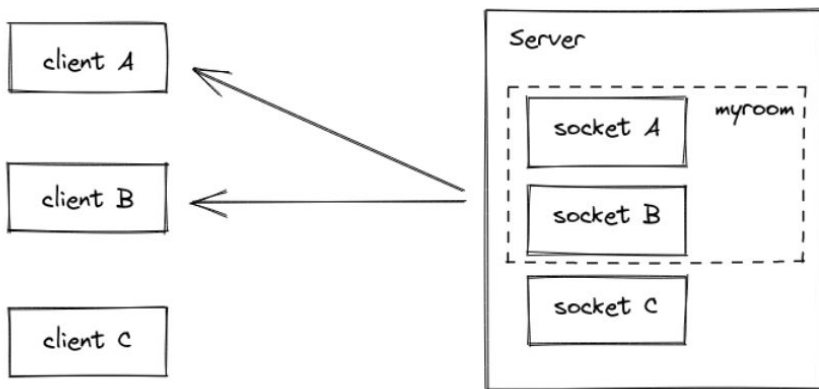
Miscellaneous

Home > Events > Rooms

Version: 4.x

Rooms

A *room* is an arbitrary channel that sockets can `join` and `leave`. It can be used to broadcast events to a subset of clients:



! INFO

Please note that rooms are a **server-only** concept (i.e. the client does not have access to the list of rooms it has joined).

Joining and leaving

Sample use cases

Disconnection

With multiple Socket.IO servers

Implementation details

Room events

Joining and leaving

Agenda

TypeScript

WebAssembly

WebSockets

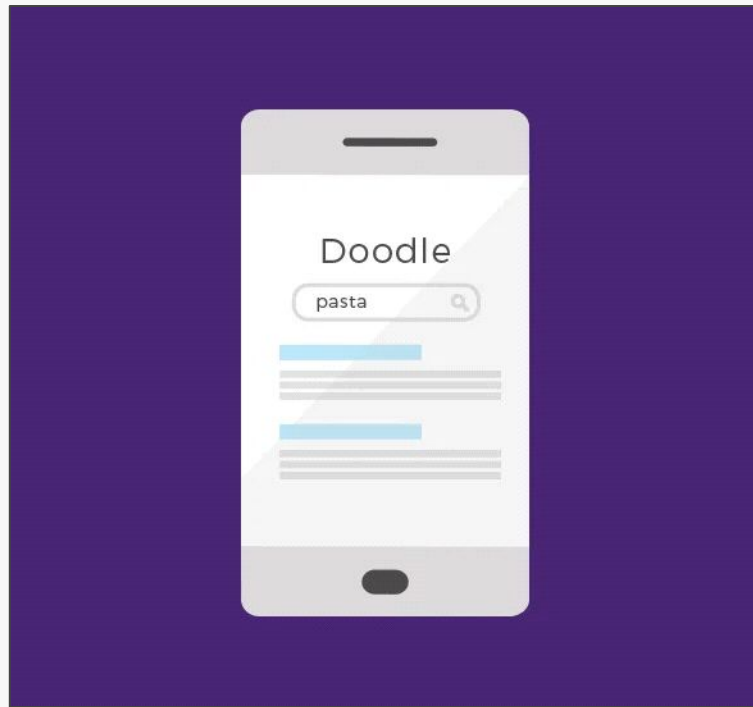
✨ **Progressive Web Apps (PWAs)**

Progressive Web Apps (PWAs)

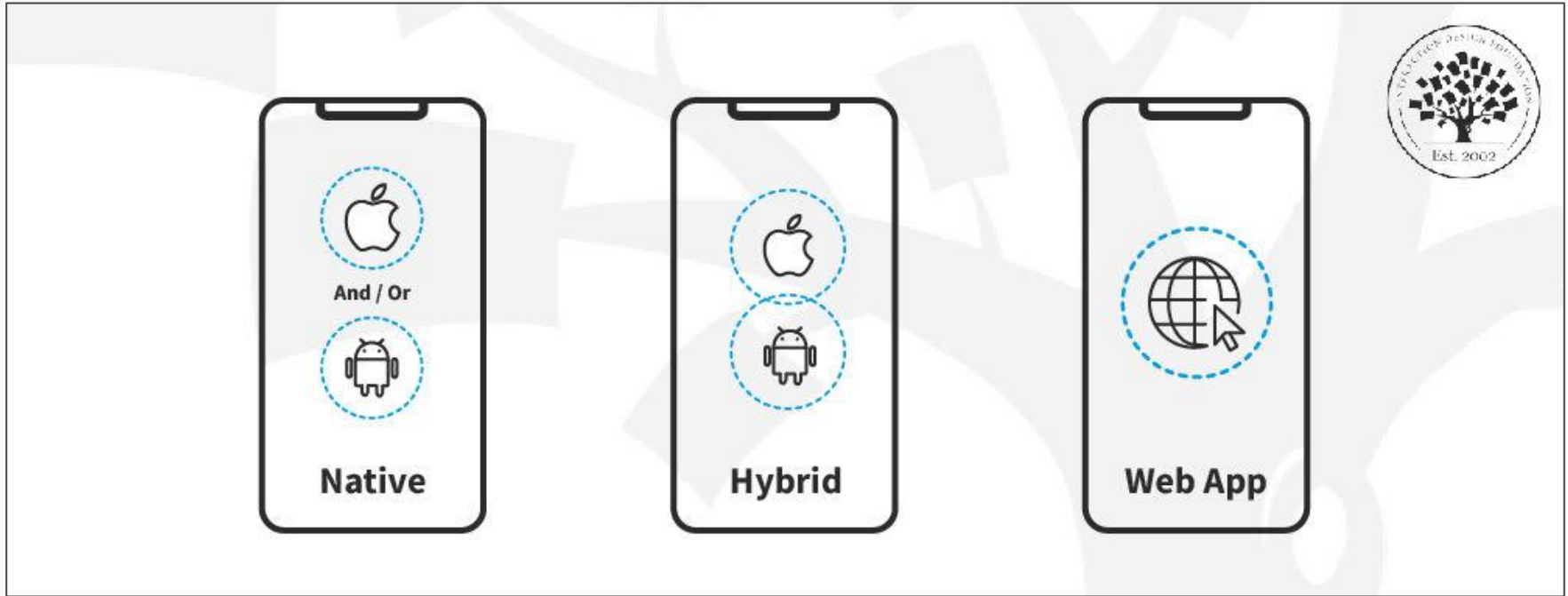
Set de técnicas que tienen por finalidad crear aplicaciones web que entregan una **experiencia similar a la de las aplicaciones móviles**

Algunas capacidades buscadas son:

- poder funcionar de forma *offline*,
- soportar notificaciones *push*,
- *caching* local de recursos,
- que sea segura,
- apariencia y velocidad casi nativas



Desarrollo de apps móviles



Desarrollo de apps móviles

Nativo

- **iOS:** swift o Objective-C (Xcode, Interface Builder)
- **Android:** Kotlin o Java (Android Studio, Android SDK)

Cross-Platform/Hybrid

- React Native (JS)
- Flutter (Dart)
- Xamarin (C#)
- Ionic (JS)
- Unity (C#)

Web based

- **Progressive Web Apps**
- Apache Cordova

Progressive Web Apps (PWAs)

Su objetivo facilitar el desarrollo de apps de alta calidad usando solo tecnologías web (más barato y más fácil)

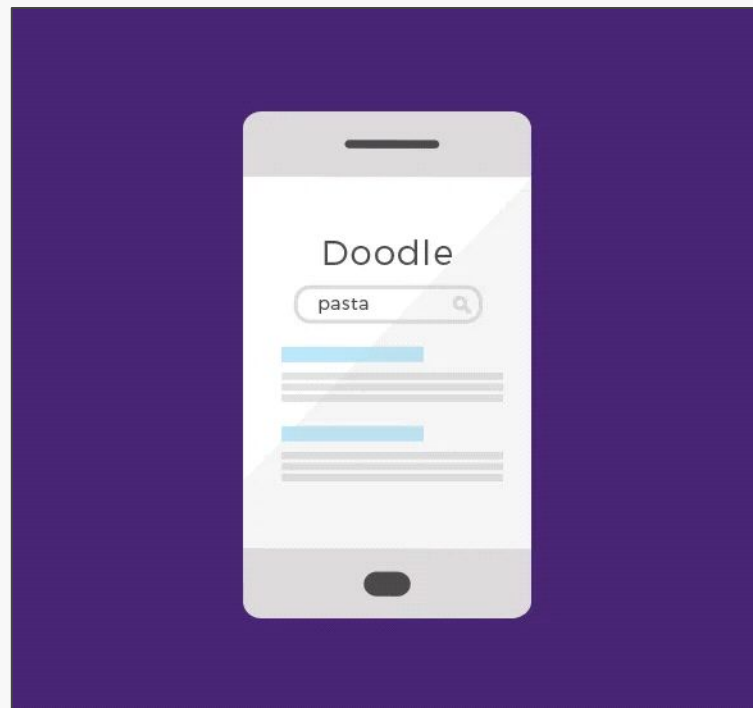
Adicionalmente, reducen la "fricción de instalación" (tener una app en el store ayuda muy poco en términos de descubribilidad, mejor que sea descubrible por buscadores)



Componentes clave

Para que una PWA funcione como tal y sume valor, existen varios elementos clave, entre los que se encuentran:

- HTTPS
- Web App Manifest
- Service workers
- Cache API
- Push notifications



¿Qué es HTTPS?

Versión segura de HTTP

HTTP sobre TLS

- **TLS** es un protocolo de seguridad que ofrece privacidad e integridad de datos para las comunicaciones en Internet
- **HTTPS** es la versión segura de HTTP, el principal protocolo utilizado para enviar datos en la web

Comunicación encriptada entre cliente y servidor

Web App Manifest

Es un archivo JSON que proporciona información sobre una aplicación (como nombre, autor, icono y descripción) en un documento. Su extensión oficial es `.webmanifest` pero puede usar `.json`

Su principal uso en el contexto de PWAs, para entregar la información que el navegador necesita para instalar la aplicación

manifest.json

```
{
  "short_name": "MDN",
  "name": "MDN Web Docs",
  "icons": [
    {
      "src": "/favicon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/favicon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ],
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#000000",
  "background_color": "#ffffff"
}
```


Web App Manifest

Es un archivo JSON que proporciona información sobre una aplicación (como nombre, autor, icono y descripción) en un documento. Su extensión oficial es `.webmanifest` pero puede usar `.json`

Su principal uso en el contexto de PWAs, para entregar la información que el navegador necesita para instalar la aplicación

app.webmanifest

```
{
  "short_name": "MDN",
  "name": "MDN Web Docs",
  "icons": [
    {
      "src": "/favicon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/favicon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ],
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#000000",
  "background_color": "#ffffff"
}
```

Web App Manifest

Una PWA requiere un manifest, ya que este archivo le indica al dispositivo que instala:

- Nombre de la app
- Ubicación de los íconos asociados
- URL de inicio (relativa al dominio)
- Orientación por defecto
- *Splash screen*

Documento detallado y estandarizado ([docs](#))

app.webmanifest

```
{
  "short_name": "MDN",
  "name": "MDN Web Docs",
  "icons": [
    {
      "src": "/favicon-192x192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "/favicon-512x512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ],
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#000000",
  "background_color": "#ffffff"
}
```

Web App Manifest

Para linkear el manifest a una aplicación, se debe incluir dentro de un tag `link`, similar a linkear una hoja de estilos de CSS

Si utilizamos múltiples páginas HTML, el manifest debe estar linkeado en todas

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <link rel="manifest" href="manifest.json" />
    <!-- ... -->
  </head>
  <body></body>
</html>
```

Web app manifest | web.dev: Qué es y cómo usar el web app manifest

[Resources](#) ▾[Baseline](#)[Blog](#)[Case Studies](#)

/



English ▾

[Sign in](#)

Resources

[Privacy](#)[Accessibility](#)[HTML](#)[Images](#)[Responsive Design](#)[Forms](#)[PWA](#)[CSS](#)[Performance](#)[Testing](#)[JavaScript](#)

Filter

[Caching](#)[Serving](#)[Workbox](#)[Offline data](#)[Installation](#)[Web app manifest](#)[Installation prompt](#)[Update](#)[Enhancements](#)[Detection](#)[OS Integration](#)[Window management](#)

Chrome is back at Google I/O on May 20-21! [Register now](#)

web.dev > Resources > PWA

Was this helpful?

Web app manifest

The web app manifest is a file you create that tells the browser how you want your web content to display as an app in the operating system. The manifest can include basic information such as the app's name, icon, and theme color; advanced preferences, such as desired orientation and app shortcuts; and catalog metadata, such as screenshots.

Warning: A web app manifest is required for your PWA to meet the [installability criteria](#) in every browser, although Chrome may still allow [installation without it](#).

Each PWA should include a single manifest per application, typically hosted in the root folder, and linked on all HTML pages your PWA can be installed from. Its official extension is `.webmanifest`, so you could name your manifest something like `app.webmanifest`.

Note: While the recommended extension is `.webmanifest`, any filename will work as long as it's delivered with the `application/`

On this page

[Adding a web app manifest to your PWA](#)[Linking to your manifest](#)[Debugging the manifest](#)[For Chromium browsers](#)[For Firefox](#)[Designing your PWA experience](#)[Basic fields](#)[Recommended fields](#)[Extended fields](#)[Promotional fields](#)[Capabilities Fields](#)[Resources](#)

Web app manifests - Progressive web apps | MDN:

Qué contiene el manifiesto de una aplicación web

Filter

background_color

categories

description

display

display_override

file_handlers

icons

id

launch_handler

name

note_taking

orientation

prefer_related_applications

protocol_handlers

related_applications

scope

Web app manifests

A **web application manifest**, defined in the [Web Application Manifest](#) specification, is a [JSON](#) text file that provides information about a web application.

The most common use for a web application manifest is to provide information that the browser needs to install a [progressive web app](#) (PWA) on a device, such as the app's name and icon.

A web application manifest contains a single JSON object where the top-level keys are called *members*.

Members

This section lists the members that may appear in the manifest.

All members are optional in the specification, but some applications require some members to be present. For example, [PWAs must provide certain manifest members](#).

- [background_color](#)
- [categories](#)
- [description](#)

In this article

Members

Example manifest

Deploying a manifest

Splash screens

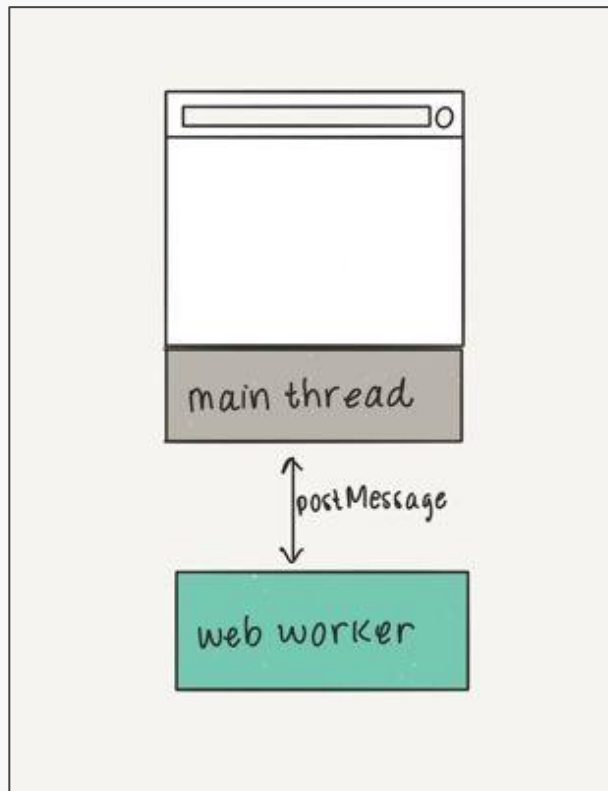
Browser compatibility

See also

Web workers y service workers

En primer lugar, según su definición, un *web worker* es utilizado para ejecutar código de JavaScript fuera del *thread* principal

En específico, un *service worker* actúa como *middleware* entre el servidor y el cliente, por lo que permiten almacenar recursos del servidor, facilitan el funcionamiento *offline* y soportan el envío de notificaciones *push* desde el servidor

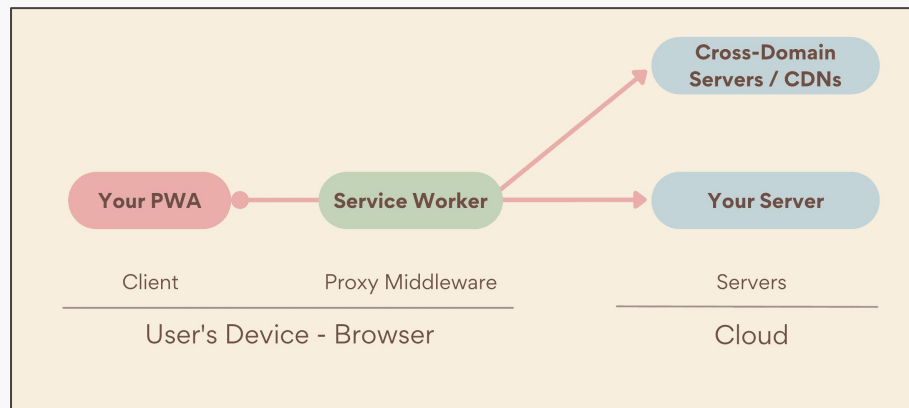


Service workers

Actúan como *proxy servers* entre las aplicaciones web, el navegador y la red (cuando está accesible)

Apuntan a permitir crear experiencias *offline* efectivas, interceptando peticiones de red y realizando la acción apropiada si la conexión de red está disponible y/o hay contenidos actualizados en el servidor

También permitirán el acceso a notificaciones tipo push y APIs de sincronización en segundo plano



Service workers

Para utilizar un *service worker*, este debe **registrarse en la PWA**. Después de verificar si el navegador es compatible con la Service Worker API, se puede registrar el *service worker*

Una vez que se registra, el *service worker* se coloca entre la PWA y la red, interceptando solicitudes y sirviendo las respuestas correspondientes

app.js

```
if ('serviceWorker' in navigator) {  
  window.addEventListener('load', () => {  
    navigator.serviceWorker  
      .register('/service-worker.js')  
      .then(registration => {  
        console.log(  
          'Service Worker registered with scope:',  
          registration.scope  
        );  
      })  
      .catch(error => {  
        console.error('Registration failed:', error);  
      });  
  });  
}
```


Service workers

Para utilizar un *service worker*, este debe registrarse en la PWA. Después de verificar si el navegador es compatible con la Service Worker API, se puede registrar el *service worker*

Una vez que se registra, el *service worker* se coloca entre la PWA y la red, interceptando solicitudes y sirviendo las respuestas correspondientes

```
service-worker.js
// Code executes in its own worker or thread
self.addEventListener('install', event => {
  console.log('Service worker installed');
});
self.addEventListener('activate', event => {
  console.log('Service worker activated');
});
```

Service workers | web.dev: Cómo registrar un *service worker* y su ciclo de vida

Resources

Privacy

Accessibility

HTML

Images

Responsive Design

Forms

PWA

CSS

Performance

Testing

JavaScript

≡ Filter

Progressive Web Apps

Getting started

Foundations

App design

Assets and data

Service workers

Caching

Serving

Workbox

Offline data

Installation

Web app manifest

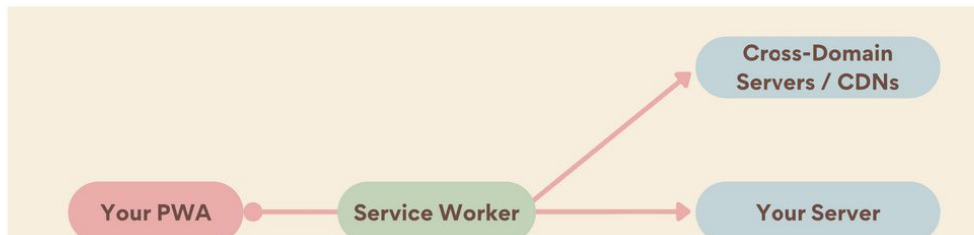
Chrome is back at Google I/O on May 20-21! [Register now](#)

web.dev > Resources > PWA

Was this helpful? 👍 🗨

Service workers

Users expect apps to start reliably on slow or flaky network connections, or even offline. They expect the content they've most recently interacted with, such as media tracks or tickets and itineraries, to be available and usable. When a request isn't possible, they expect the app to tell them instead of silently failing or crashing. And they want all of this to happen quickly. As you can see in [Milliseconds make millions](#), even a 0.1 second improvement in load times can improve conversion by up to 10%. Service workers are the tool that lets your Progressive Web App (PWA) live up to your users' expectations.



On this page

[Register a service worker](#)

Verify whether a service worker is registered

Scope

Lifecycle

Update a service worker

Service worker lifespan

Capabilities

Resources

Service Worker API: Documentación en MDN sobre la Service Worker API

Filter

Service Worker API

Guides

Using Service Workers

Interfaces

Cache

CacheStorage

Client

Clients

ExtendableEvent

ExtendableMessageEvent

FetchEvent

InstallEvent

NavigationPreloadManager

ServiceWorker

ServiceWorkerContainer

Service Worker API

Note: This feature is available in [Web Workers](#).

Service workers essentially act as proxy servers that sit between web applications, the browser, and the network (when available). They are intended, among other things, to enable the creation of effective offline experiences, intercept network requests, and take appropriate action based on whether the network is available, and update assets residing on the server. They will also allow access to push notifications and background sync APIs.

Service worker concepts and usage

A service worker is an event-driven [worker](#) registered against an origin and a path. It takes the form of a JavaScript file that can control the web page/site that it is associated with, intercepting and modifying navigation and resource requests, and caching resources in a very granular fashion to give you complete control over how your app behaves in certain situations (the most obvious one being when the network is not available).

Service workers run in a worker context: they therefore have no DOM access and run on a different thread to the main JavaScript that powers your app. They are non-blocking and designed to be fully

In this article

Service worker concepts and usage

Other use case ideas

Interfaces

Specifications

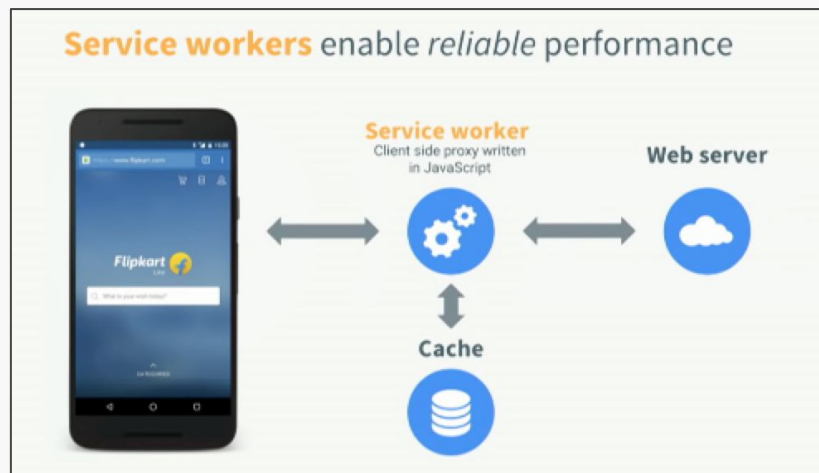
See also

El service worker es lo más importante

Un service worker es similar a un [web worker](#) (proceso que ejecuta *scripts* JavaScript en un *thread* secundario), operando de manera independiente de la aplicación a la que está asociada

Puede recibir mensajes incluso cuando el navegador está cerrado, permaneciendo *idle* cuando no se utiliza y reiniciando cuando es necesario

Un *service worker* actúa como capa de caché de solicitudes HTTP y *handler* de mensajes push

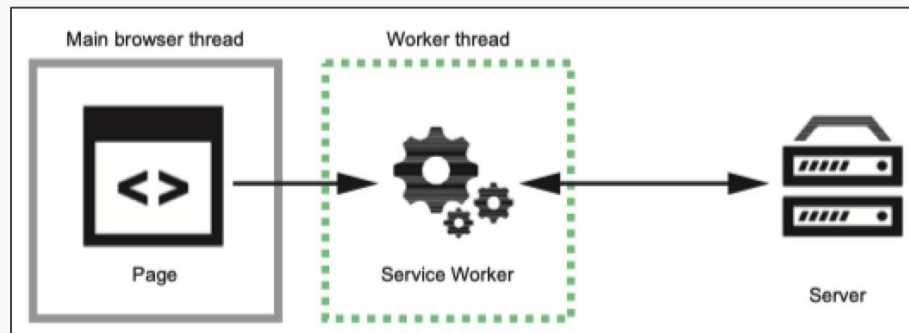


Service worker como intermediario

En la primera visita al sitio el service worker es instalado, y visitas posteriores a las distintas páginas del sitio van a llamar al service worker

Una vez instalado el *worker* es independiente de la ventana del browser o de un tab y escucha y actúa sobre eventos que ocurren en páginas bajo su control

Por ejemplo una solicitud puede ser interceptada, modificada, traspasada o devuelta

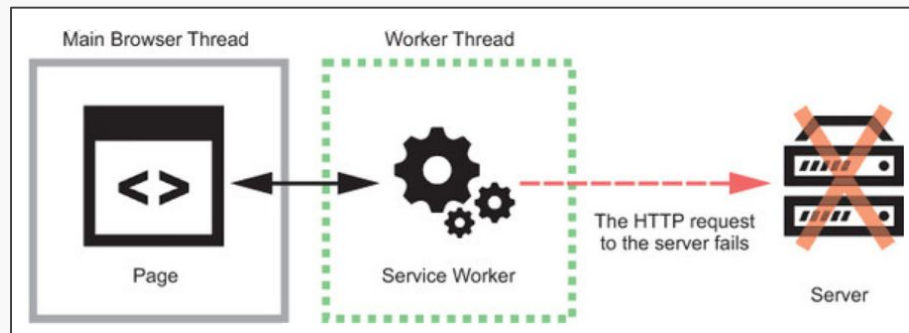


Service worker como intermediario

En la primera visita al sitio el service worker es instalado, y visitas posteriores a las distintas páginas del sitio van a llamar al service worker

Una vez instalado el *worker* es independiente de la ventana del browser o de un tab y escucha y actúa sobre eventos que ocurren en páginas bajo su control

Por ejemplo una solicitud puede ser interceptada, modificada, traspasada o devuelta



Ciclo de vida del service worker

Registration

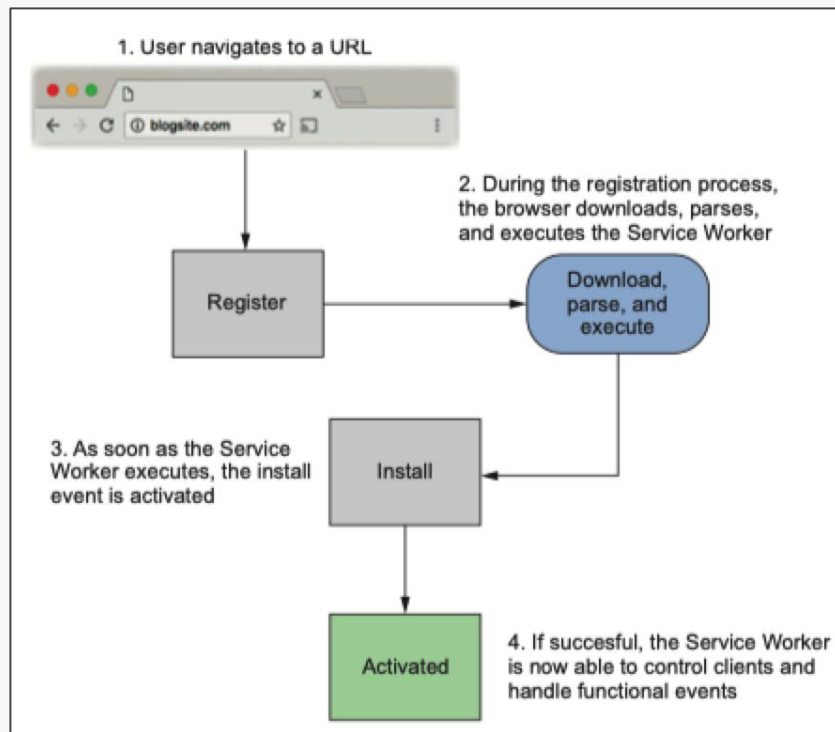
Informa al browser donde está y comienza instalación en el background. Informa sobre el scope (por defecto archivos y subarchivos de la carpeta del SW). Solo ocurre la primera vez o si es actualizado

Installation

Si el browser determina que no ha sido registrado antes o que ha sufrido cambios. Se prepara al SW para ser usado: inicialización del caché, la UI y *assets* estáticos

Activation

Habilita al SW para responder a futuras cargas de páginas



Ciclo de vida del service worker

Registration

Informa al browser donde está y comienza instalación en el background. Informa sobre el scope (por defecto archivos y subarchivos de la carpeta del SW). Solo ocurre la primera vez o si es actualizado

Installation

Si el browser determina que no ha sido registrado antes o que ha sufrido cambios. Se prepara al SW para ser usado: inicialización del caché, la UI y *assets* estáticos

Activation

Habilita al SW para responder a futuras cargas de páginas

```
self.addEventListener('install', event => {  
  console.log('Service Worker: Instalado');  
});  
self.addEventListener('activate', event => {  
  console.log('Service Worker: Activado');  
});  
self.addEventListener('fetch', event => {  
  console.log('Service Worker: Fetching',  
    event.request.url);  
});
```


Ciclo de vida del service worker

Registration

Informa al browser donde está y comienza instalación en el background. Informa sobre el scope (por defecto archivos y subarchivos de la carpeta del SW). Solo ocurre la primera vez o si es actualizado

Installation

Si el browser determina que no ha sido registrado antes o que ha sufrido cambios. Se prepara al SW para ser usado: **inicialización del caché**, la UI y *assets* estáticos

Activation

Habilita al SW para responder a futuras cargas de páginas

```
let CACHE_NAME = 'my-cache';
let urlsCache = [
  'css/style.css',
  'images/myIcon.png',
  'scripts/index.js'
];

self.addEventListener('install',
function(event) {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(function(cache) {
        console.log('Opened cache');
        return cache.addAll(urlsCache);
      })
  );
});
```

Service workers tutorial — Uploadcare Blog: Tutorial de cómo implementar *service workers*

Uploadcare 

[Uploader](#)

[Image CDN](#)

[Pricing](#)

[Developers](#) 

[Company](#) 

[Log in](#)

[Sign up](#)

[← Blog](#)

FRONT-END

Service workers tutorial

Last edited: January 6, 2023



Nikola Đuza

Web developers or anyone involved in web development can tell you they know of “workers” in browsers. However, the problem is that most folks think of either web workers in general, or service workers (or even shared workers) when they say “workers.” These terms can be confusing, especially if you didn’t come across them.

Progressive Web Apps in 100 Seconds // Build a PWA from Scratch:

Breve introducción al funcionamiento de las PWA



CL

Search



Sign in



Progressive Web Apps in 100 Seconds // Build a PWA from Scratch



Fireship



3.38M subscribers

Subscribe



39K



Share



Save



781K views · 3 years ago #100SecondsOfCode #pwa #webdev



Next.js in 100 Seconds // Plus



Full Beginner's Tutorial

Fireship



1.1M views · 3 years ago



7 Web Features You Didn't Know



Existed

Fireship



277K views · 3 years ago



Intro To Service Workers &



Caching

Traversy Media



236K views · 6 years ago



What Is A Progressive Web App



(PWA) - Why Is It A Game...

Going Headless with John

39K views · 2 years ago



10 regrets of experienced



programmers

Fireship



1.4M views · 5 months ago



Progressive Web Apps (PWAs)



vs native mobile apps | Leigh...

Element Engineering Australia

4.5K views · 5 months ago

Agenda

TypeScript

WebAssembly

WebSockets

Progressive Web Apps (PWA)

✨ **¿Qué se viene pronto?**

Próximas fechas importantes: clases y ayudantías

	Clases y ayudantías						
	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
Semana 12 (19/5 - 25/5)		Charla GraphQL (Daniela Flores)	Día de las Glorias Navales	Almacenamiento (cookies, tokens, IndexDB, etc)			
Semana 13 (26/5 - 01/6)		HOY		Testing	Sala de ayuda: E1		
Semana 14 (02/6 - 08/6)		Seguridad web		Protocolos seguros			
Semana 15 (09/6 - 15/6)		Privacidad		Ética en CS			
Semana 16 (16/6 - 22/6)		Aplicaciones en el mundo real		Charla TBD	Día Nacional de los Pueblos Indígenas		

Próximas fechas importantes: evaluaciones

	Evaluaciones							
	Lunes	Martes	Miércoles	Jueves	Viernes		Sábado	Domingo
Semana 12 (19/5 - 25/5)	Control 3		Día de las Glorias Navales					
Semana 13 (26/5 - 01/6)		HOY			Fin E1	Inicio E2		
Semana 14 (02/6 - 08/6)	Control 4							
Semana 15 (09/6 - 15/6)					Fin E2	Inicio E3		
Semana 16 (16/6 - 22/6)	Control 5				Día Nacional de los Pueblos Indígenas			

Clase 17

Tecnologías adicionales

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl